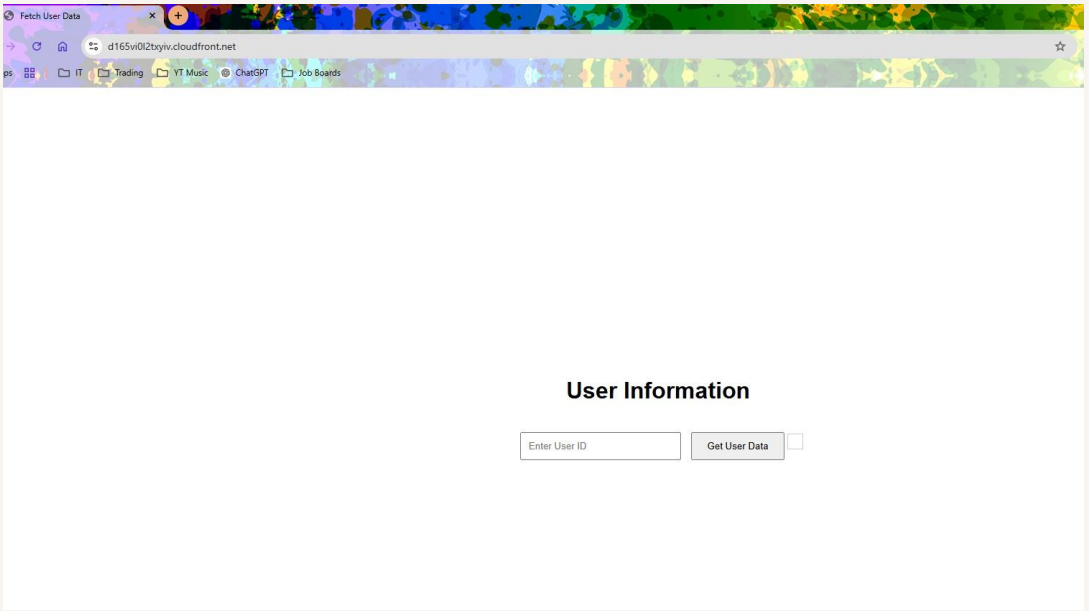
Website Delivery with CloudFront

By,

Melvin green



Introducing Today's Project!

In this project, I demonstrate the use of Amazon CloudFront. The goal is to deepen my understanding of content delivery networks (CDNs) and to learn how to configure the presentation tier within a three-tier architecture.

Tools and concepts

Services I used were Amazon S3 and Amazon CloudFront. Key concepts I learned include content delivery networks (CDNs), distributions, origin access control, performance and load times, and S3 static website hosting versus CloudFront.

Project reflection

This project took me approximately 3 hours to complete. The most challenging part was understanding origin access control. The most rewarding part was comparing S3 to CloudFront in terms of website performance.

I chose to do this project today because I am learning to build a three-tier app, and these are some of the initial steps in building the presentation tier of a three-tier app.

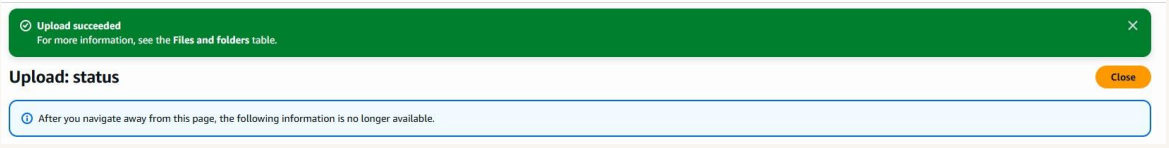
Set Up S3 and Website Files

I began the project by creating an S3 bucket to store files. CloudFront isn't used for this task because it's designed for content delivery, not file storage.

The three files that make up my website are: index.html, the main file for the website, where I organize the text, images, and other elements that make up the webpage; style.css, which defines the visual appearance of the HTML elements, controlling everything from font sizes and colors to layout, helping maintain a consistent style across the site; and script.js, a JavaScript file that adds interactivity to the website, containing the

instructions for actions such as elements moving or changing when a button is clicked or a form is submitted.

I validated that my website files work by opening them on my local computer.

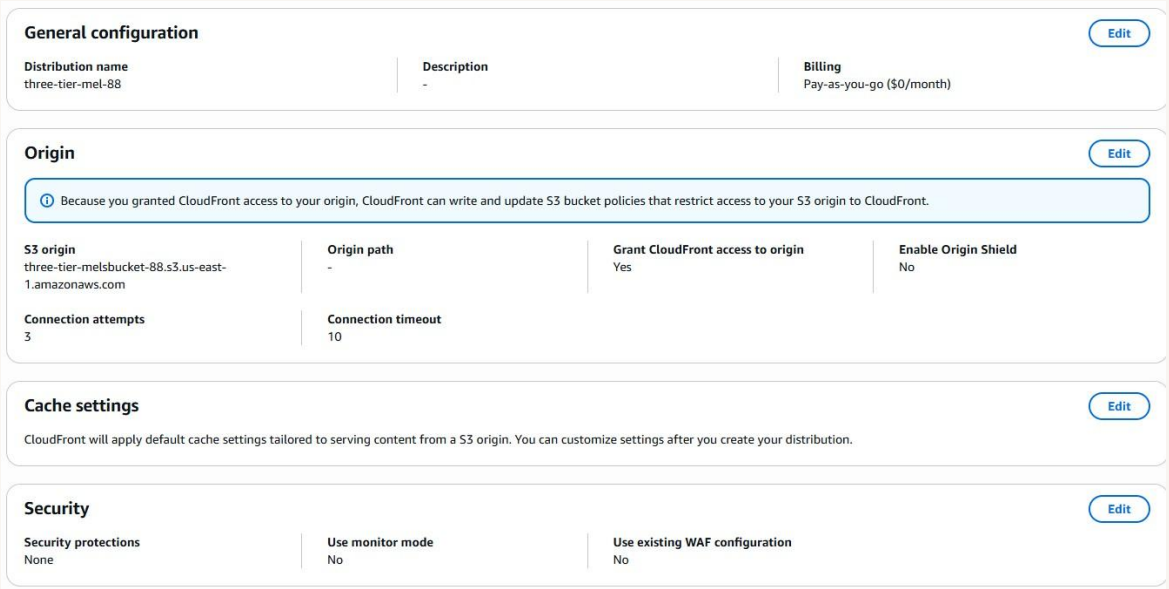


Exploring Amazon CloudFront

Amazon CloudFront is a content delivery network that speeds up the distribution of static and dynamic web content, such as .html, .css, .js, and image files. Businesses and developers use CloudFront to cache their website content across multiple servers around the world. When a user requests content served through CloudFront, the request is routed to the edge location that provides the lowest latency (time delay), so the content is delivered with the best possible performance.

To use Amazon CloudFront, I set up a distribution, which is a set of instructions that tells CloudFront how to deliver content. I selected the "single website or app" distribution type, with the origin set to the S3 bucket I created earlier.

My CloudFront distribution's default root object is index.html. This means when a user visits my website's root URL, they will see the content described in index.html.



Handling Access Issues

When I tried visiting my distributed website, I ran into an access denied error because I hadn't yet given CloudFront permission to access my S3 bucket. By default, S3 buckets are private, and CloudFront needs explicit permission to access the files in the bucket.

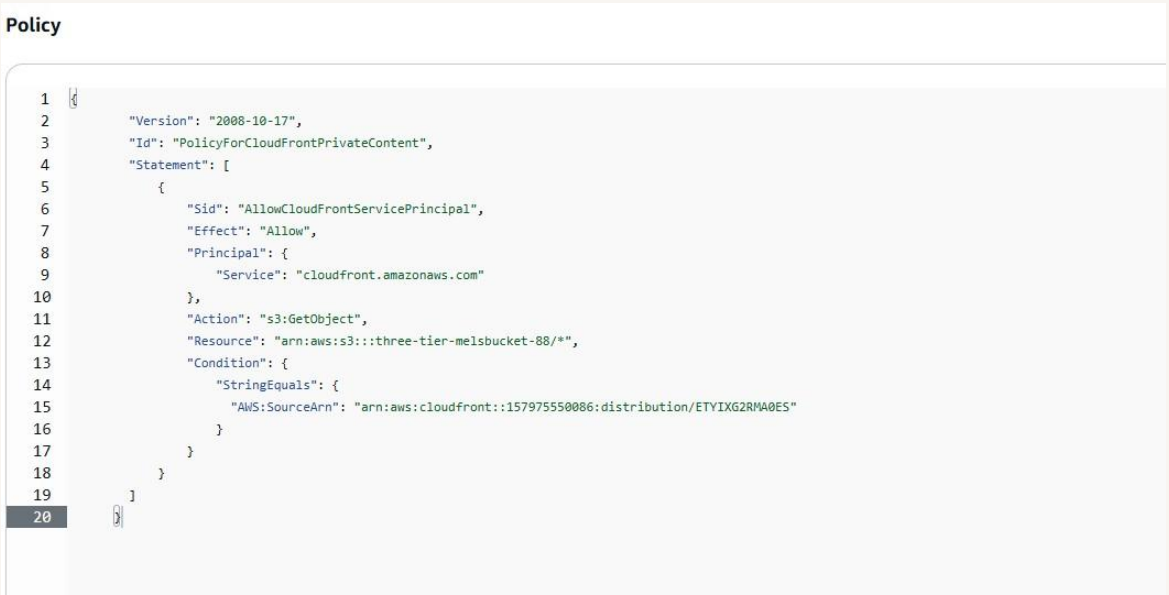
My distribution's origin access settings were set to "public," which could give anyone direct access to the content from the origin. However, this caused the "Access Denied" error because my S3 bucket objects are private by default. To fix this, I need to update my S3 bucket settings to grant CloudFront access to the objects.

To resolve the error, I set up Origin Access Control (OAC). OAC acts as a special identity for CloudFront that prevents direct public access to the S3 bucket. It allows me to keep my S3 bucket and objects private, while still ensuring they can be accessed through CloudFront.

Updating S3 Permissions

Once I set up my OAC, I still needed to update my bucket policy because OAC alone doesn't grant CloudFront permission to access my bucket — it only identifies CloudFront as the requester. For the restricted access to take effect, the S3 bucket policy must explicitly grant that OAC permission to access the bucket's contents.

Creating an OAC automatically gives me a policy I could copy, which grants my CloudFront permission to access my S3 Bucket



S3 vs CloudFront for Hosting

For my project extension, I compare S3 with CloudFront. I initially ran into an access error because I enabled S3 static website hosting without making my objects publicly available.

Here's the corrected version: I tried resolving this by turning off "Block Public Access" in my S3 bucket, but I still ran into a 403 Forbidden error when accessing the static website URL. This was because turning off Block Public Access alone doesn't grant public access. I still needed to update the bucket policy to explicitly allow public access to the objects.

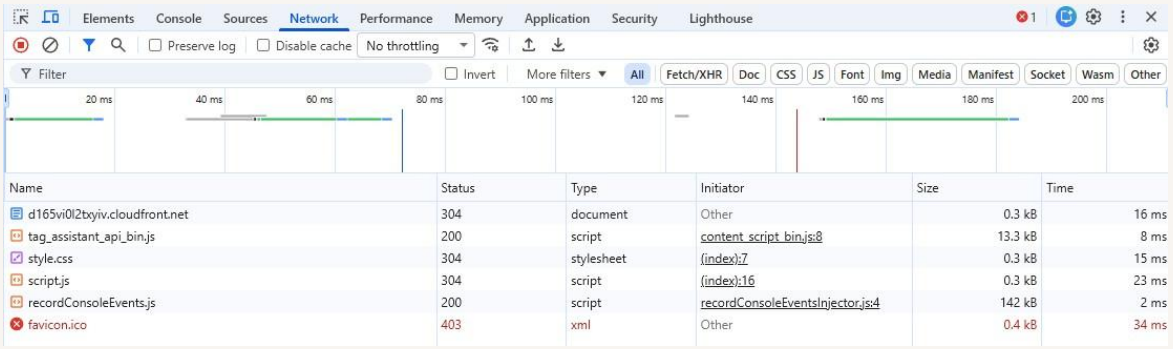
I could finally see my S3-hosted website after updating my bucket policy to allow the public to access the bucket's objects. This worked because my bucket policy had previously only allowed CloudFront access, so a change in S3 permissions was required.

Compared to the permission settings for my CloudFront distribution, using S3 static website hosting meant I had to enable public access to my bucket. I preferred using CloudFront because it meant bucket access was restricted to CloudFront only, which is better for security.

S3 vs CloudFront Load Times

Load time refers to how long it takes for the browser to receive requested content and display it to the user. The load times for the CloudFront site were faster than the S3 site because content is cached from edge locations.

A business would prefer CloudFront when performance and load times are important to them. S3 static website hosting might be sufficient when hosting a simple website without strict performance or load time requirements.


Elements
Console
Sources
Network
Performance
Memory
Application
Security
Lighthouse
Preserve log
Disable cache
No throttling
Filter
Invert
More filters
All
Fetch/XHR
Doc
CSS
JS
Font
Img
Media
Manifest
Socket
Wasm
Other
20 ms
40 ms
60 ms
80 ms
100 ms
120 ms
140 ms
160 ms
180 ms
200 ms
Name
Status
Type
Initiator
Size
Time
d165vi0l2txyiv.cloudfront.net 304 document Other 0.3 kB 16 ms
tag_assistant_api_bin.js 200 script content_script_bin.js:8 13.3 kB 8 ms
style.css 304 stylesheet (index):7 0.3 kB 15 ms
script.js 304 script (index):16 0.3 kB 23 ms
recordConsoleEvents.js 200 script recordConsoleEventsInjector.js:4 142 kB 2 ms
favicon.ico 403 xml Other 0.4 kB 34 ms